

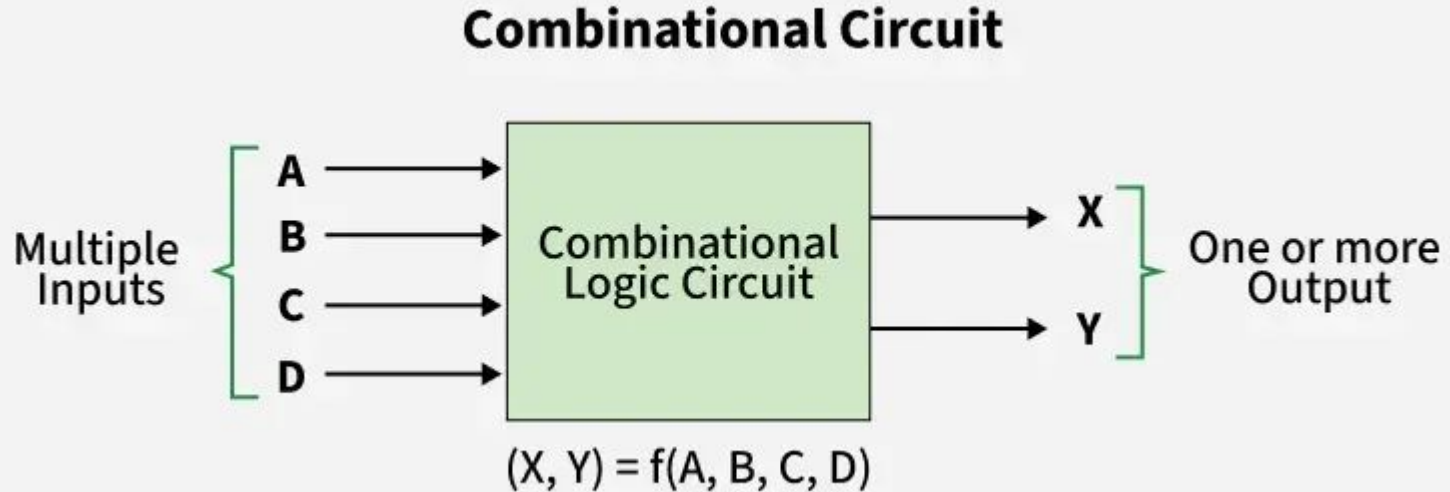
EEE283

Digital Logic Design

Course Instructor: A K M Anindya Alam

Topic 5:
**Combinational Logic Circuits for Non-Arithmetic
Operations**

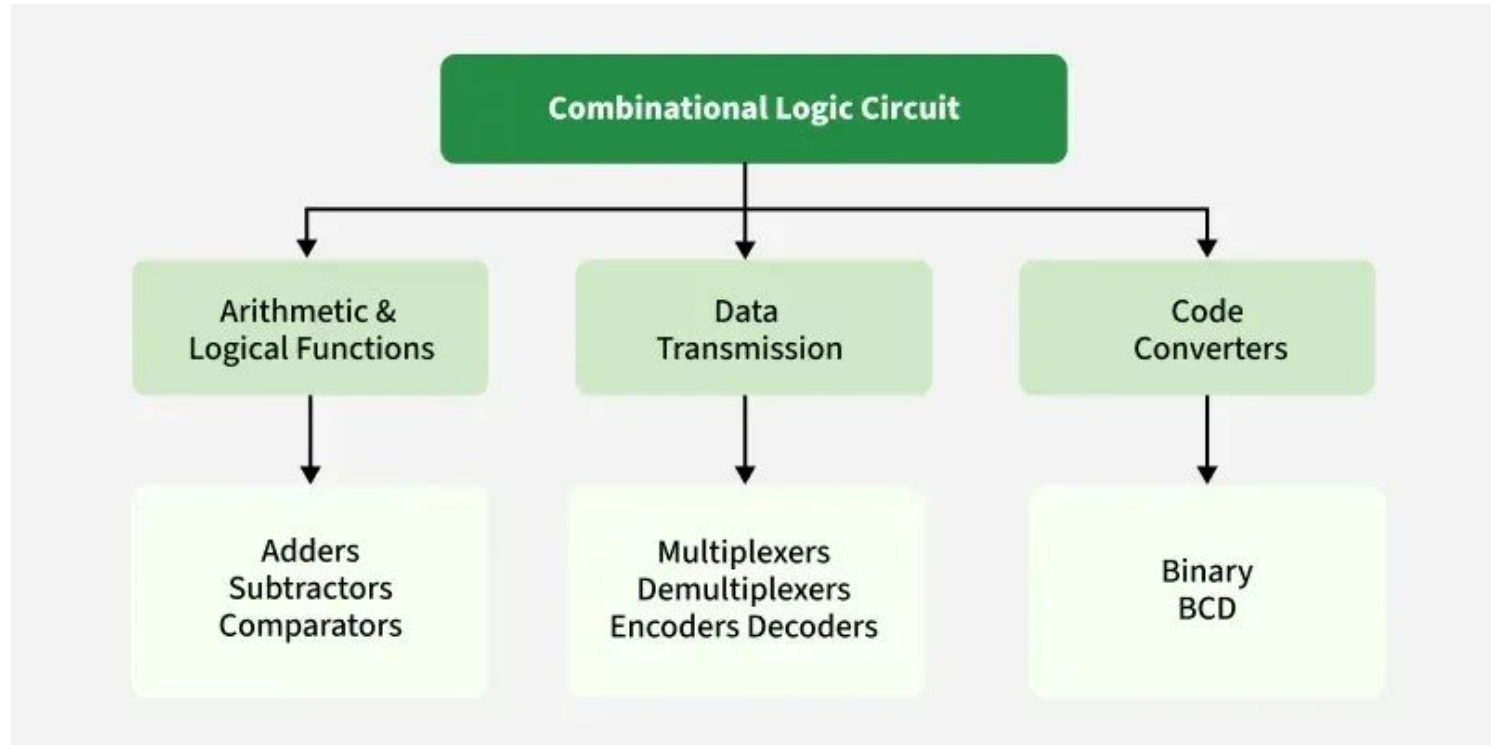
Combinational Circuits



Combinational Circuits

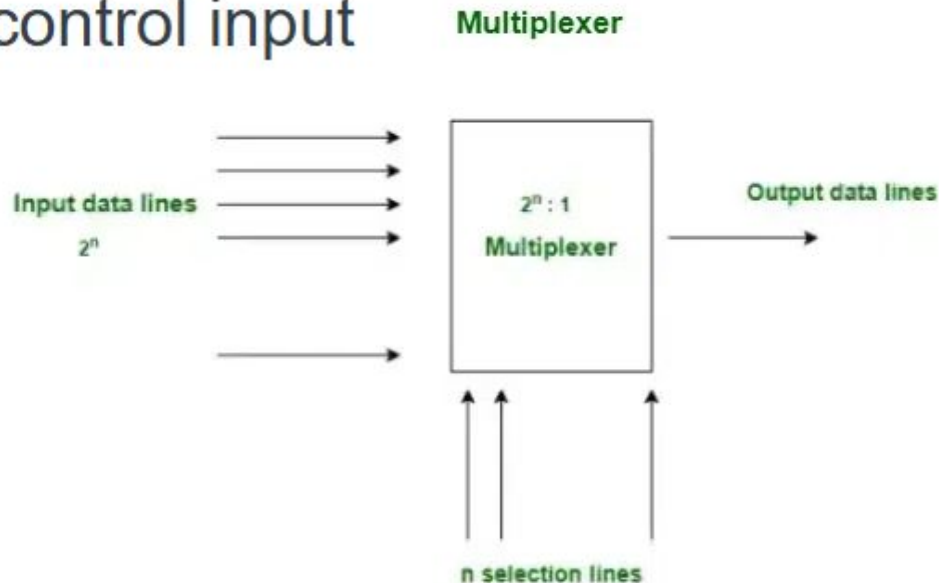
- It only cares about the present input and state.
- They are specially designed using multiple interconnected logic gates such that the output will be generated by computing the logical combinations of the present input only.
- No clock pulse is present here; moreover, no previously stored value or state is taken into consideration here.
- The output is independent of previous states.

Classification of Combinational Circuits



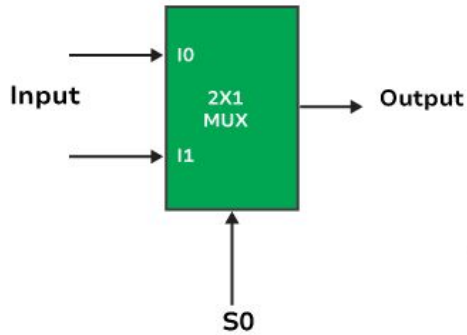
Multiplexer

- Selects between one of N inputs to connect to output
- $\log_2 N$ -bit select input – control input



2:1 Multiplexer

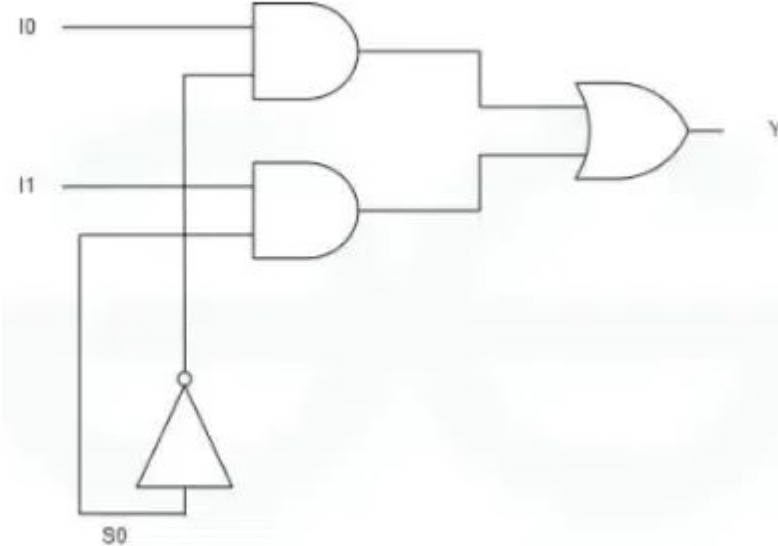
2:1 Multiplexer



Truth Table

S_0	I_0	I_1	Y
0	0	X	0
0	1	X	1
1	X	0	0
1	X	1	1

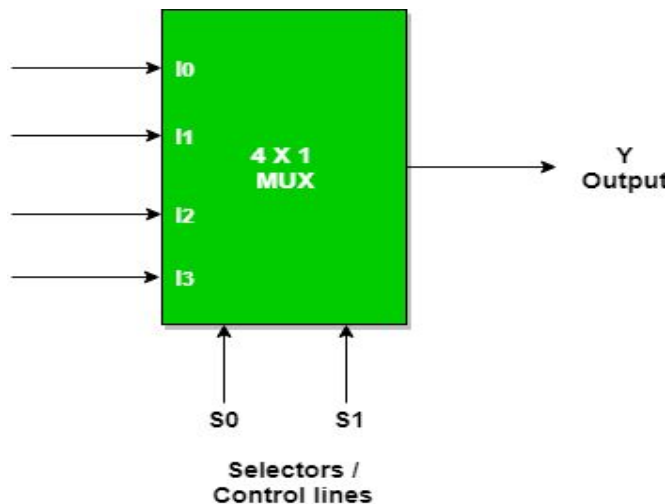
$$Y = \overline{S_0}I_0 + S_1I_1$$



The output of the 2x1 Mux will depend on the selection line S0,

- When S is 0(low), the I0 is selected
- when S0 is 1(High), I1 is selected

4:1 Multiplexer



Truth Table

S_0	S_1	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

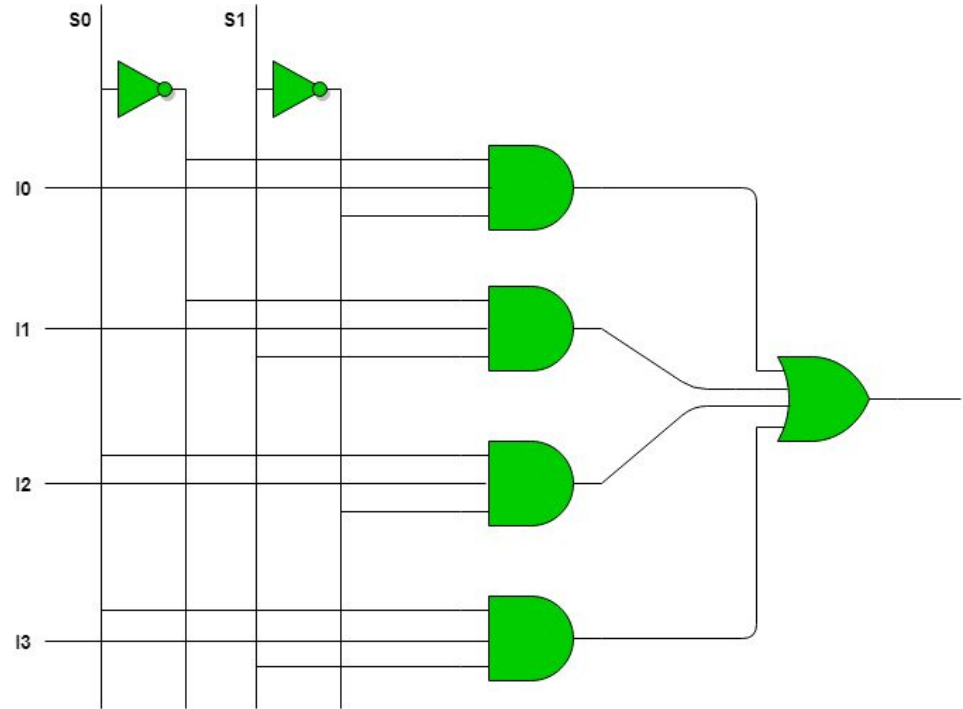
So, final equation,

$$Y = S_0'.S_1'.I_0 + S_0'.S_1.I_1 + S_0.S_1'.I_2 + S_0.S_1.I_3$$

4:1 Multiplexer

The output of the multiplexer is determined by the binary value of the selection lines

- When $S_1S_0=00$, the input I_0 is selected.
- When $S_1S_0=01$, the input I_1 is selected.
- When $S_1S_0=10$, the input I_2 is selected.
- When $S_1S_0=11$, the input I_3 is selected.



Implementation of Truth Tables Using MUX

To implement any truth table using a 2x1 MUX

x	y	f	$f \rightarrow y$
0	0	0	$f = y$
0	1	1	
1	0	1	$f = 1$
1	1	1	

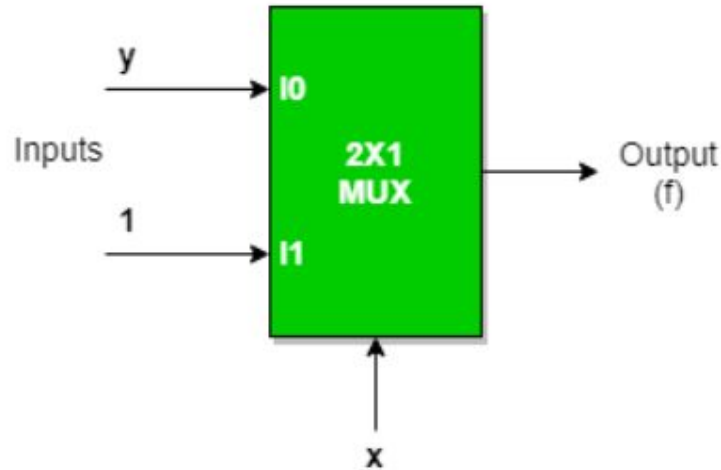
**Selection
Line**

**Input Data
Lines**

- For a truth table with 2 or less inputs you will need a 2x1 mux.
- Choose any of the table inputs as the selector line. For our example, we choose x.
- Express the output in-terms of the remaining table inputs, 0 and 1 for two cases: $x = 0$ and $x = 1$. Set the input-values of the mux accordingly.
- What will happen if there are three inputs?

Implementation of AND Gate

Given Below is the Diagram for the Logical Representation of OR gate using 2 : 1 Mux

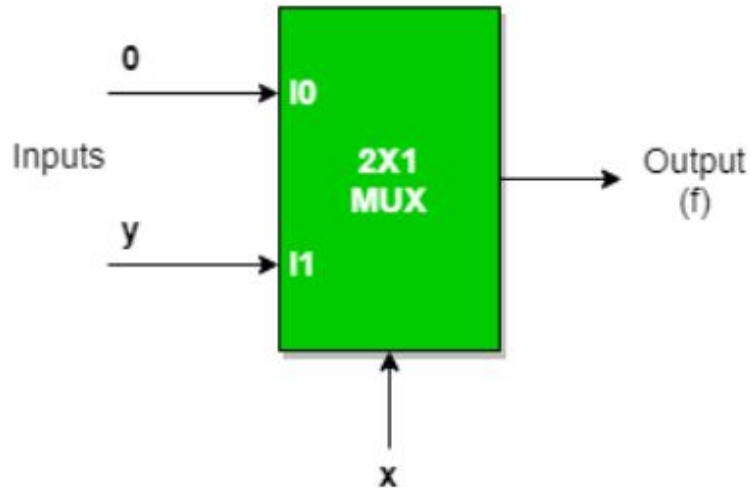


Truth Table

x	y	f	$f \rightarrow y$
0	0	0	f = y
0	1	1	
1	0	1	f = 1
1	1	1	

Implementation of AND Gate

Given Below is the Diagram for the Logical Representation of AND gate using 2 : 1 Mux

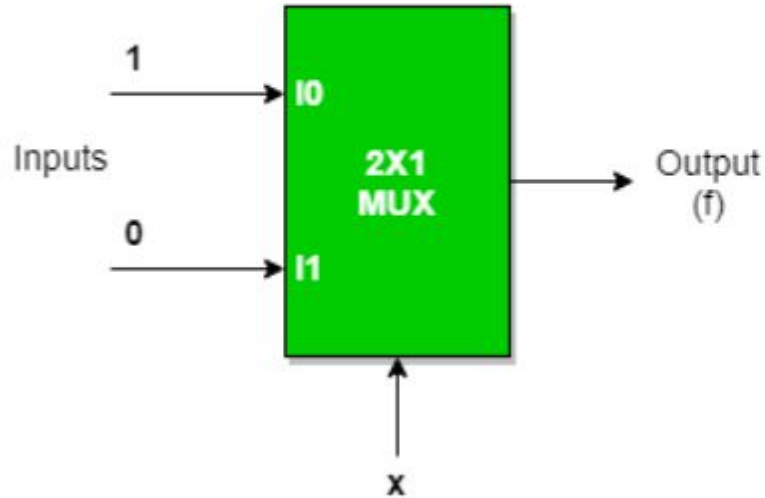


Truth Table

x	y	f	$f \rightarrow y$
0	0	0	$f = 0$
0	1	0	
1	0	0	$f = y$
1	1	1	

Implementation of NOT Gate

Given Below is the Diagram for the Logical Representation of NOT gate using 2 : 1 Mux

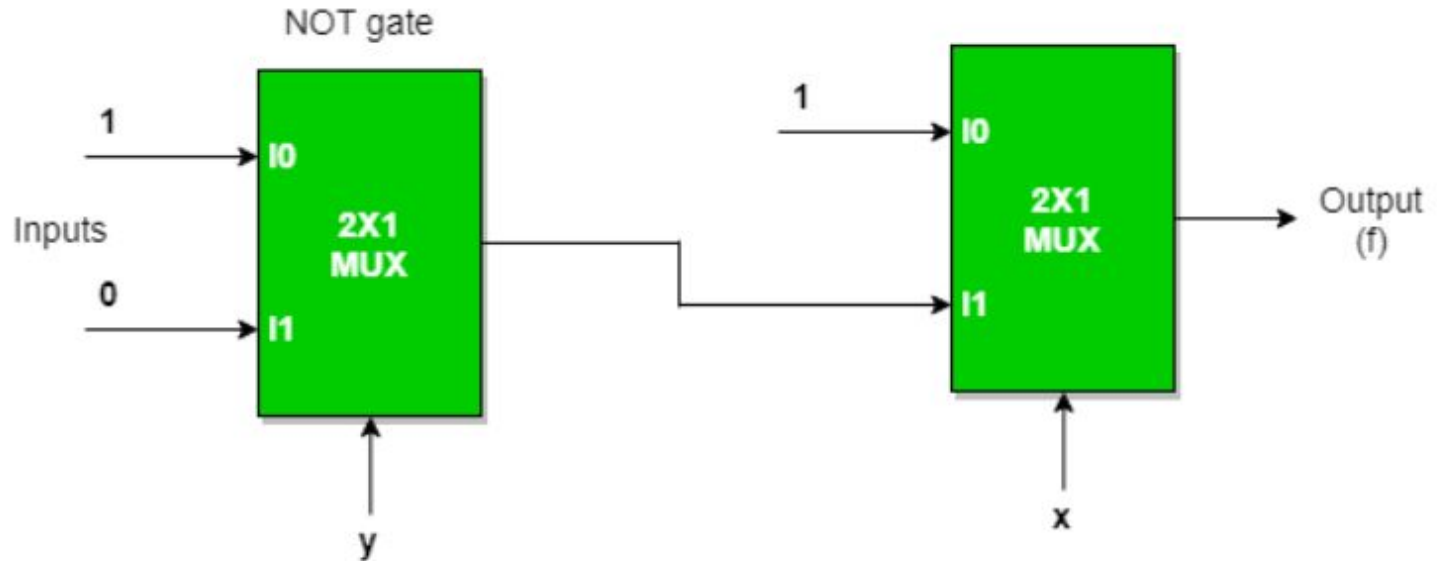


Truth Table

x	f
0	1
1	0

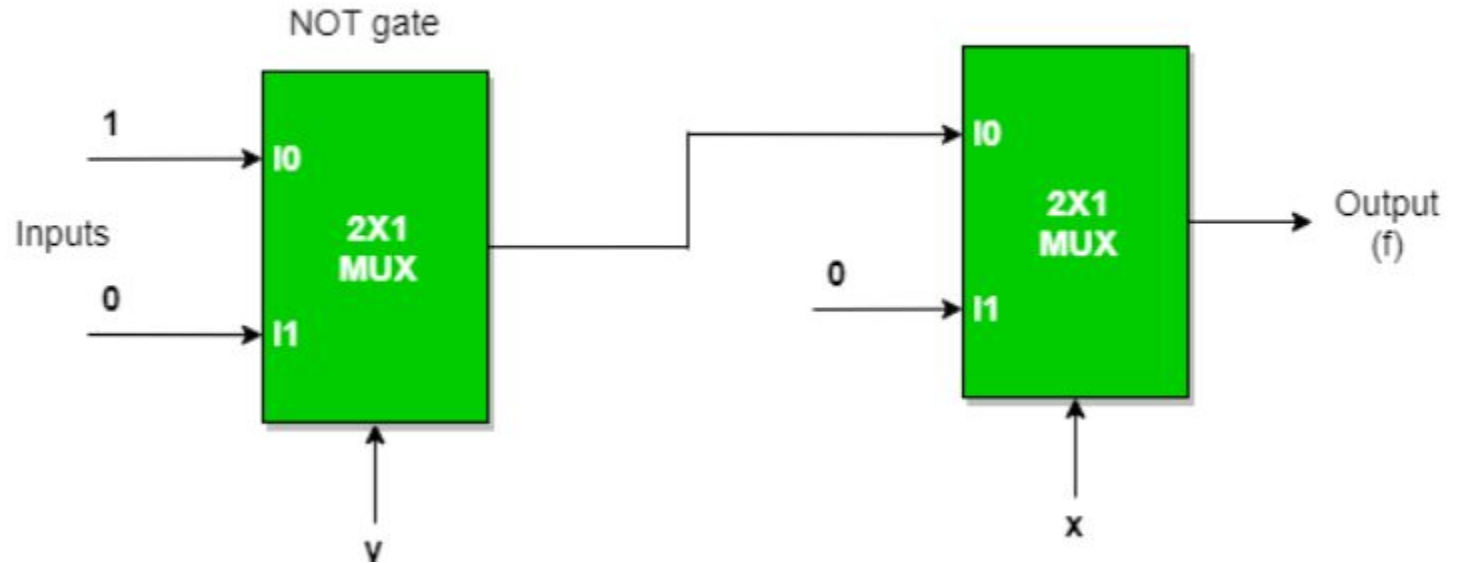
Implementation of NAND Gate

Given Below is the Diagram for the Logical Representation of NAND gate using 2 : 1 Mux



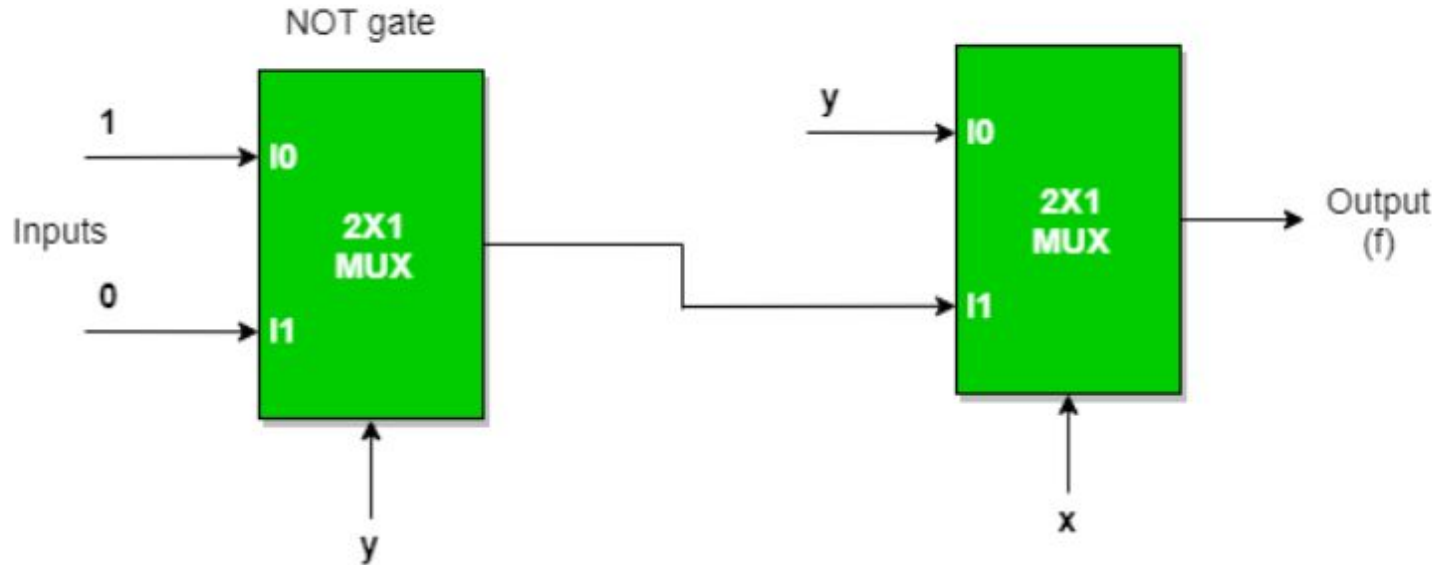
Implementation of NOR Gate

Given Below is the Diagram for the Logical Representation of NOR gate using 2 : 1 Mux

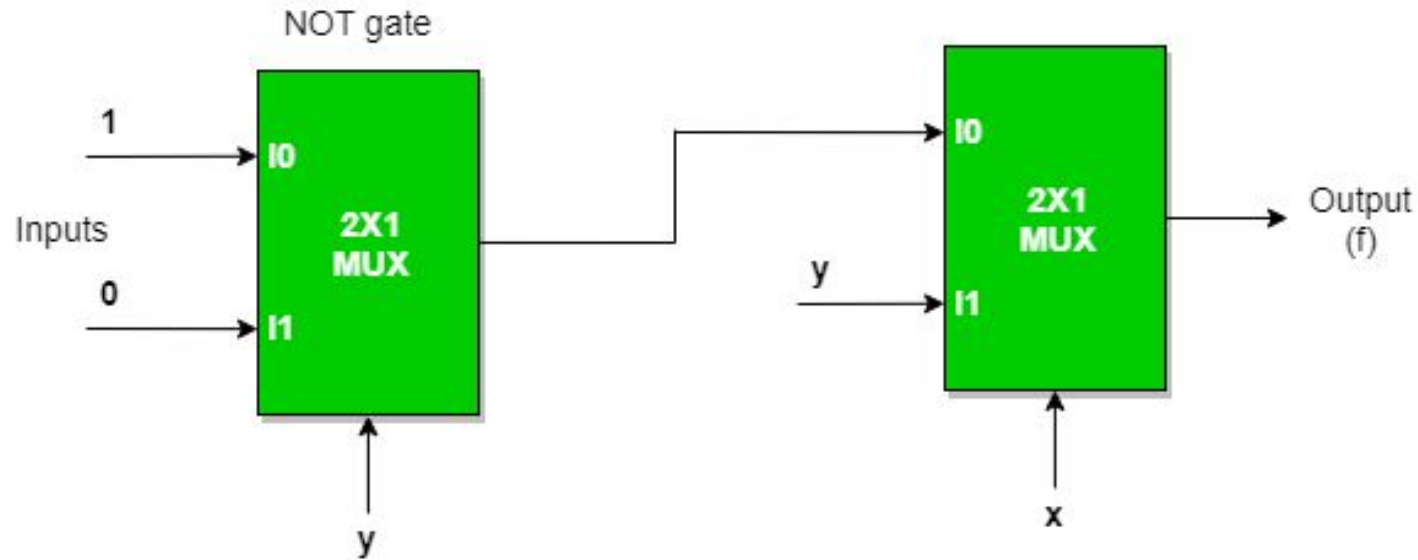


Implementation of XOR Gate

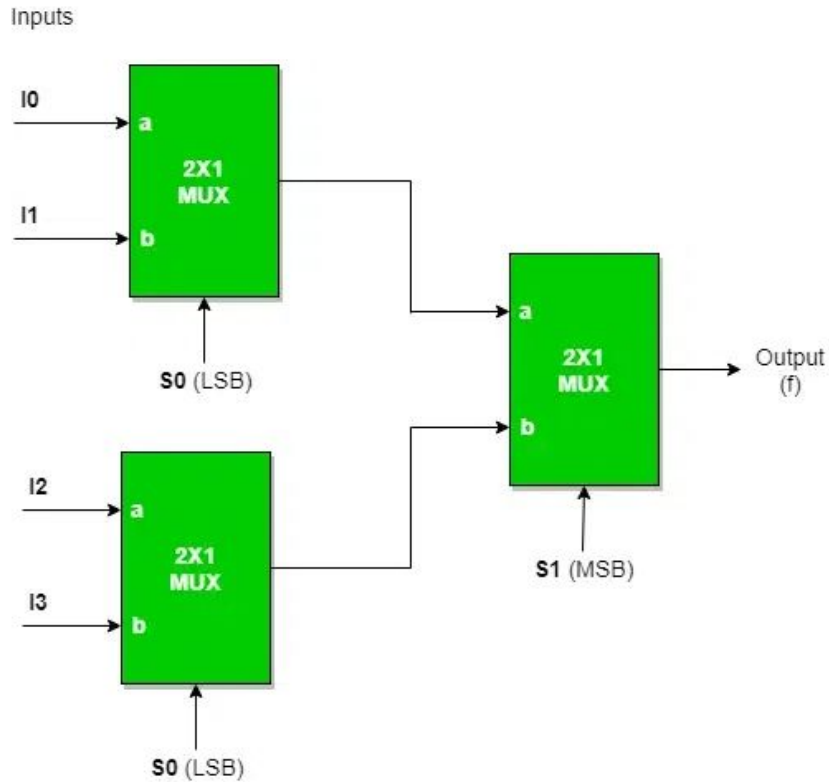
Given Below is the Diagram for the Logical Representation of EX-OR gate using 2 : 1 Mux



Implementation of XNOR Gate



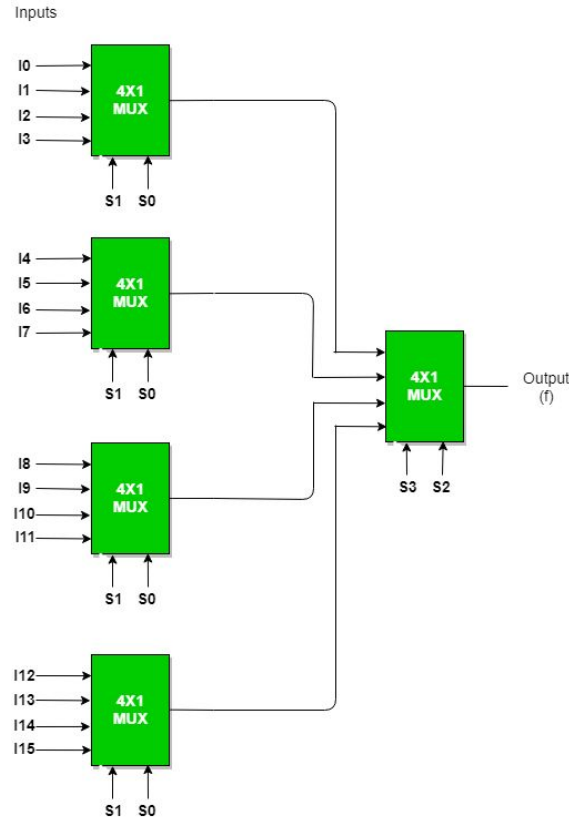
Implementation of 4:1 Mux using a 2:1 Mux



Truth Table

S_1	S_0	f
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

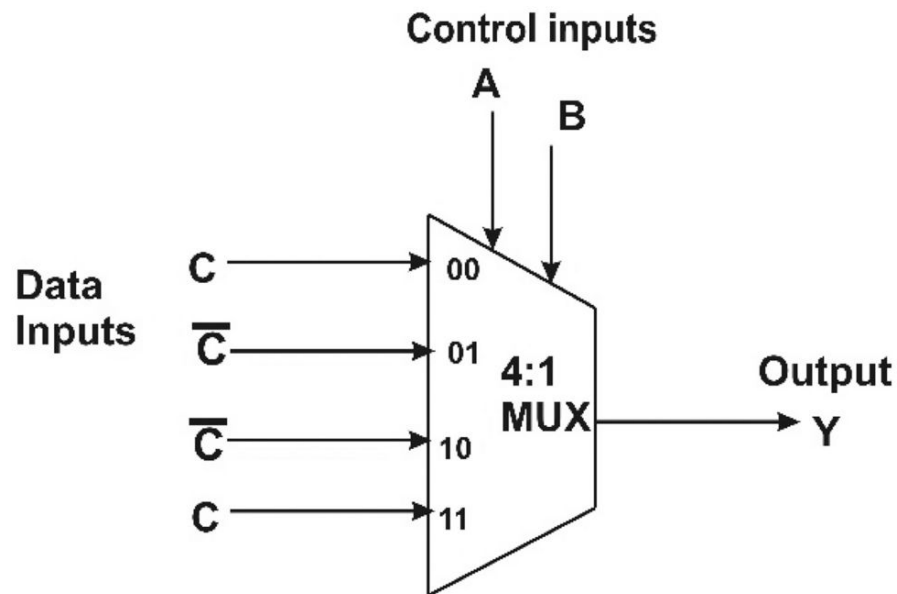
Implementation of 16:1 Mux using a 4:1 Mux



Implementation of a Three Input Logic Gate

Truth Table

Input A	Input B	Input C	$X = A \oplus B \oplus C$
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1



Try to implement this using a 2:1 MUX.

Shannon's Expansion Theorem

Any Boolean function $f(w_1, \dots, w_n)$ can be written in the form

$$f(w_1, w_2, \dots, w_n) = \overline{w_1} \cdot f(0, w_2, \dots, w_n) + w_1 \cdot f(1, w_2, \dots, w_n)$$

This expansion can be done in terms of any of the n variables.

$f(0, w_2, \dots, w_n)$ – **cofactor** of f wrt $\overline{w_1}$, $f_{\overline{w_1}}$

$f(1, w_2, \dots, w_n)$ – **cofactor** of f wrt w_1 , f_{w_1}

These cofactors are the INPUTs to the 2×1 MUX

Shannon's Expansion Theorem

Shannon's expansion can be done in terms of more than one variable.

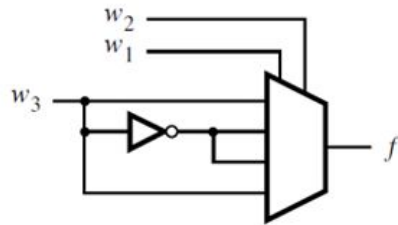
$$\begin{aligned} f(w_1, \dots, w_n) = & \bar{w}_1 \bar{w}_2 \cdot f(0, 0, w_3, \dots, w_n) + \bar{w}_1 w_2 \cdot f(0, 1, w_3, \dots, w_n) \\ & + w_1 \bar{w}_2 \cdot f(1, 0, w_3, \dots, w_n) + w_1 w_2 \cdot f(1, 1, w_3, \dots, w_n) \end{aligned}$$

w_1	w_2	w_3	f
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

$$\begin{aligned} f &= \bar{w}_1 \bar{w}_2 w_3 + \bar{w}_1 w_2 \bar{w}_3 + w_1 \bar{w}_2 \bar{w}_3 + w_1 w_2 w_3 \\ &= \bar{w}_1 \bar{w}_2 f(0,0,w_3) + \bar{w}_1 w_2 f(0,1,w_3) + w_1 \bar{w}_2 f(1,0,w_3) + w_1 w_2 f(1,1,w_3) \end{aligned}$$

$$f = \bar{w}_1 \bar{w}_2 w_3 + \bar{w}_1 w_2 \bar{w}_3 + w_1 \bar{w}_2 \bar{w}_3 + w_1 w_2 w_3$$

Each of the co-factors are the inputs to the MUX



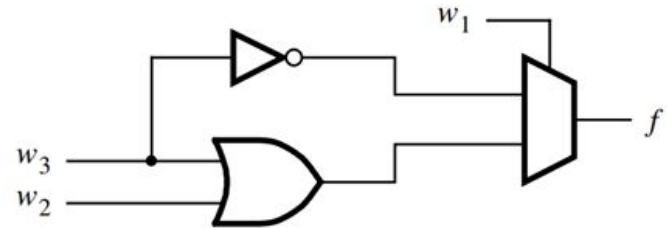
Shannon's Expansion Theorem

$$f = \bar{w}_1 \bar{w}_3 + w_1 w_2 + w_1 w_3$$

Implement the Boolean function using 2×1 MUX and 4×1 MUX using Shannon's Expansion Theorem

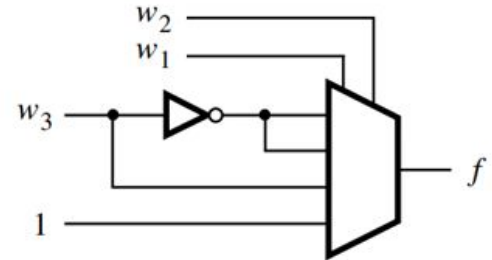
2×1 MUX

$$\begin{aligned} f &= \bar{w}_1 f_{\bar{w}_1} + w_1 f_{w_1} \\ &= \bar{w}_1 (\bar{w}_3) + w_1 (w_2 + w_3) \end{aligned}$$



4×1 MUX

$$\begin{aligned} f &= \bar{w}_1 \bar{w}_2 f_{\bar{w}_1 \bar{w}_2} + \bar{w}_1 w_2 f_{\bar{w}_1 w_2} + w_1 \bar{w}_2 f_{w_1 \bar{w}_2} + w_1 w_2 f_{w_1 w_2} \\ &= \bar{w}_1 \bar{w}_2 (\bar{w}_3) + \bar{w}_1 w_2 (\bar{w}_3) + w_1 \bar{w}_2 (w_3) + w_1 w_2 (1) \end{aligned}$$



Advantages and Disadvantages of Mux

Advantages of MUX

Some advantages of MUX are:

- MUX is cheaper circuit and can be used to implement other circuits.
- MUX is flexible circuit that can be easily implemented.

Disadvantages of MUX

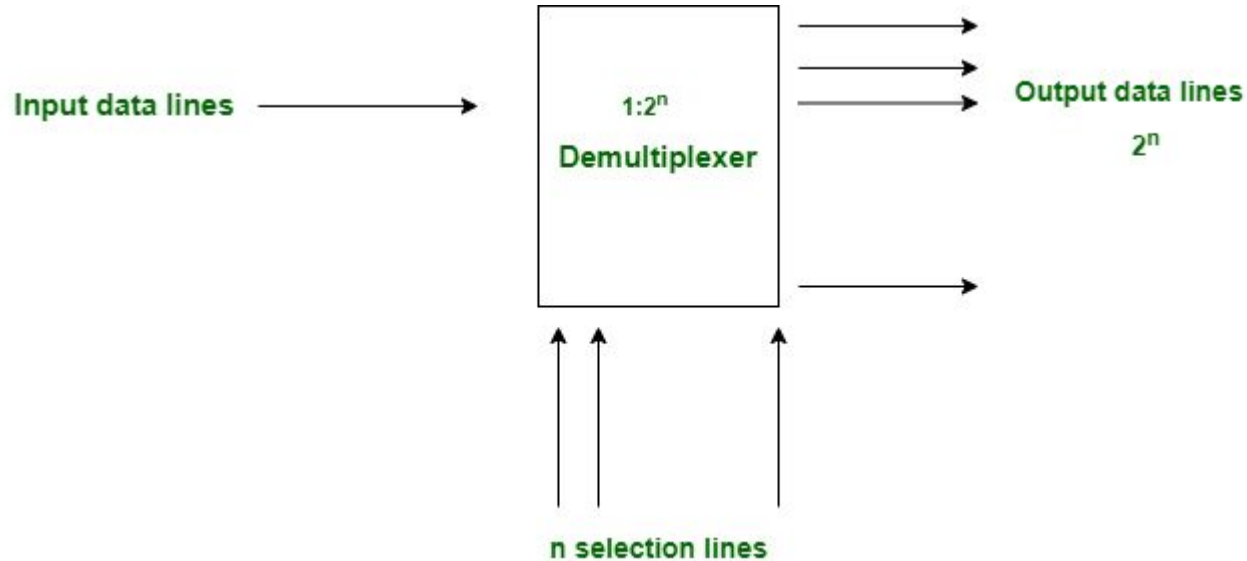
Some disadvantages of MUX are:

- MUX adds some delays in the circuits.
- The data source is limited in MUX.

Applications of MUX

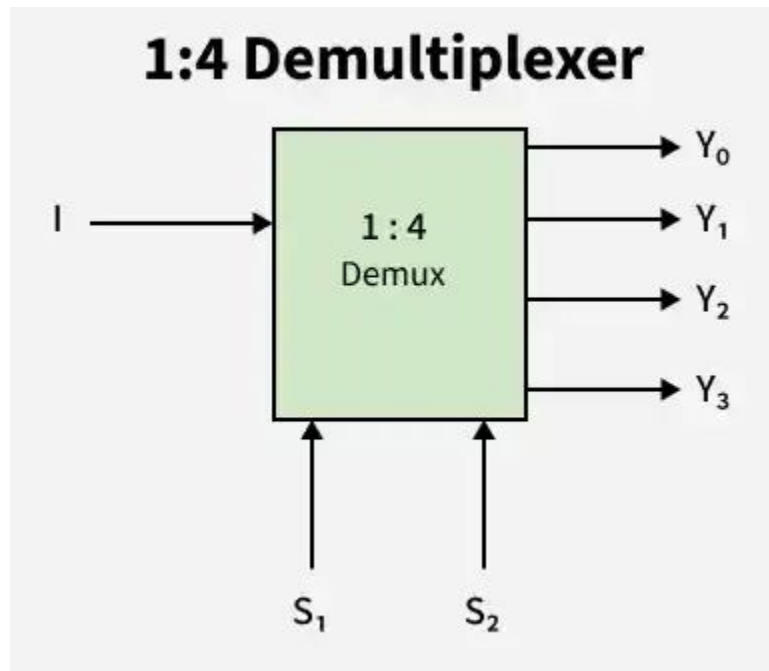
- **Data Routing** : The Mux is used for data routing in the digital system where they select one of the several data lines and re-route it the output.
- **Data Selection** : The Mux is used for data selection where they select data source according to the select lines.
- **Analog-to-Digital Conversion** : The Mux are used in [ADC](#) to select different analog input channels.
- **Address Decoding** : The Mux are used in [Microprocessors](#) or memory for address decoding.
- **Logic Function Implementation** : Mux can be used to implement various logic functions.

Demultiplexer



Opposite of a MULTIPLEXER

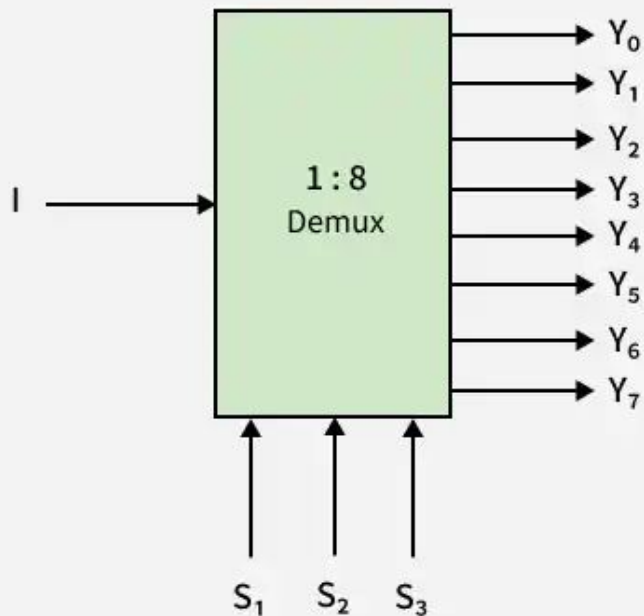
1:4 Demultiplexer



Selection Inputs		Outputs			
S_1	S_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	I
0	1	0	0	I	0
1	0	0	I	0	0
1	1	I	0	0	0

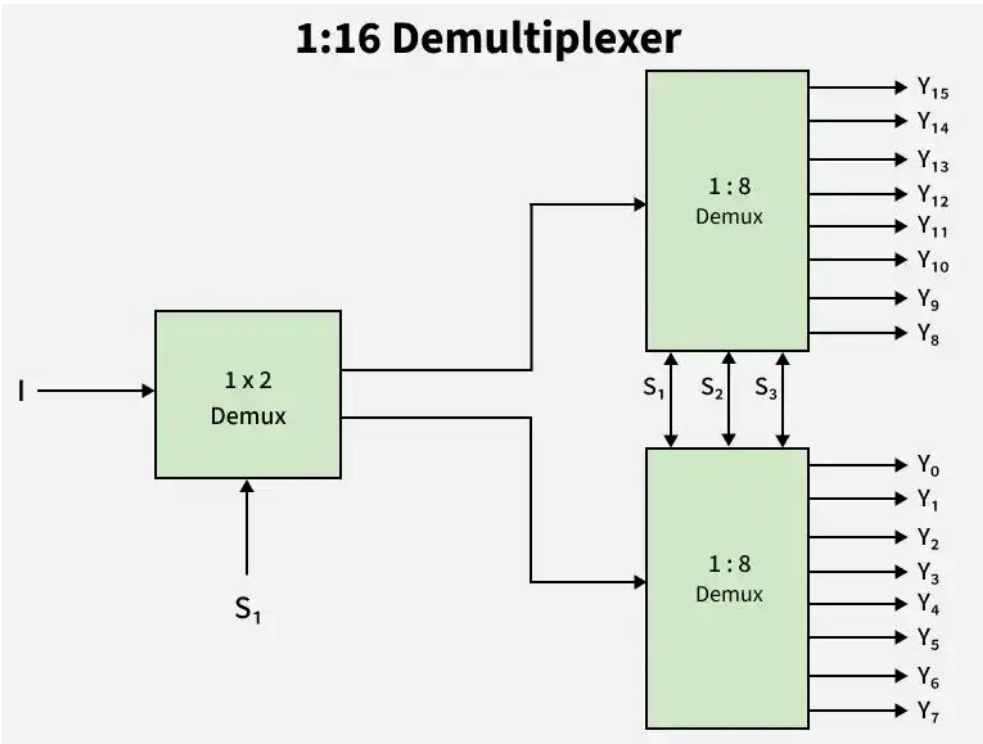
1:8 Demultiplexer

1:8 Demultiplexer



Selection Inputs			Outputs							
S_2	S_1	S_0	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

1:16 Demultiplexer



Find the truth table for this demux. Try to implement this using 4:1 demultiplexers.

Advantages and Disadvantages of Demux

Advantages of the DEMUX

- The DEMUX increases the efficiency of the particular communication system as it takes data from a specific input source and distributes it to different sources.
- The DEMUX helps to separate the different signals from the mixed data sources. Then it distributes these data to different sources.
- DEMUX can decode the signal outputs of the [multiplexer](#), as the system works in a reverse way of the MUX.

Disadvantages of the DEMUX

- The DEMUX may cause a wastage of bandwidth as it distributes the refined data in different channels. These channels can overlap with each other which leads to the loss of signal.
- The DEMUX may cause problems in the synchronization of signals. The data channels can overlap with each other which leads to the delay in the whole process.

Applications of DEMUX

- DEMUXs are majorly used to design the ALU circuits and parallel data segments.
- The DEMUXs convert the output of the MUX into the actual input. On the receiver end, the DEMUX can be used to verify the original form of the data to carry out the entire communication process.
- DEMUX helps to save the output. The output generally saved to the ALU. The output will be saved in the registers and the various storage units of the system.
- Each DEMUX has a connection with multiple registers which helps to store the processed data into it.
- The counterpart of the DEMUX regulates the data signal at the output stage of the DEMUX operation. These data can be retrieved later to read out parallelly.
- In the audio or video system, the DEMUX distributes them in different channels. In broadcasting, the DEMUX separates the composite signal into an individual one.

Lookup Table (LUT)

- A Lookup Table (LUT) is a memory-based implementation of Boolean logic
- Functionally equivalent to a $2^n \times 1$ multiplexer
- Input variables act as select lines
- MUX data inputs are fixed logic values (0 or 1)
- Output is obtained by selecting the stored value

Application of LUTs

- Eliminates complex gate-level design
- Any Boolean function can be implemented
- Logic is stored as truth-table values
- Reconfigurable without hardware changes
- Fast and predictable delay

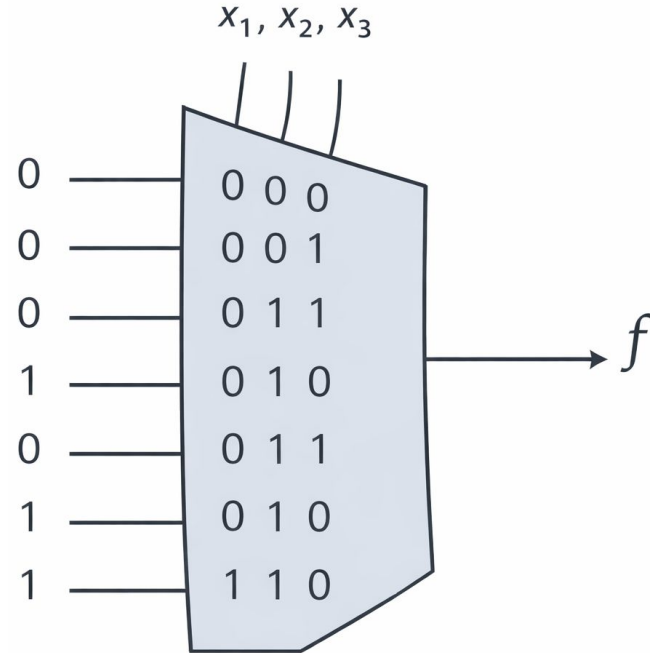
Implementation of LUTs as MUXs

- For n input variables $\rightarrow$ use a $2^n \times 1$ MUX
- Inputs $(x_1, x_2, \dots, x_n) \rightarrow$ select lines
- Data inputs $\rightarrow$ function values from truth table
- Example: 3-variable function $\rightarrow 8 \times 1$ MUX
- Each MUX input is tied to 0 or 1

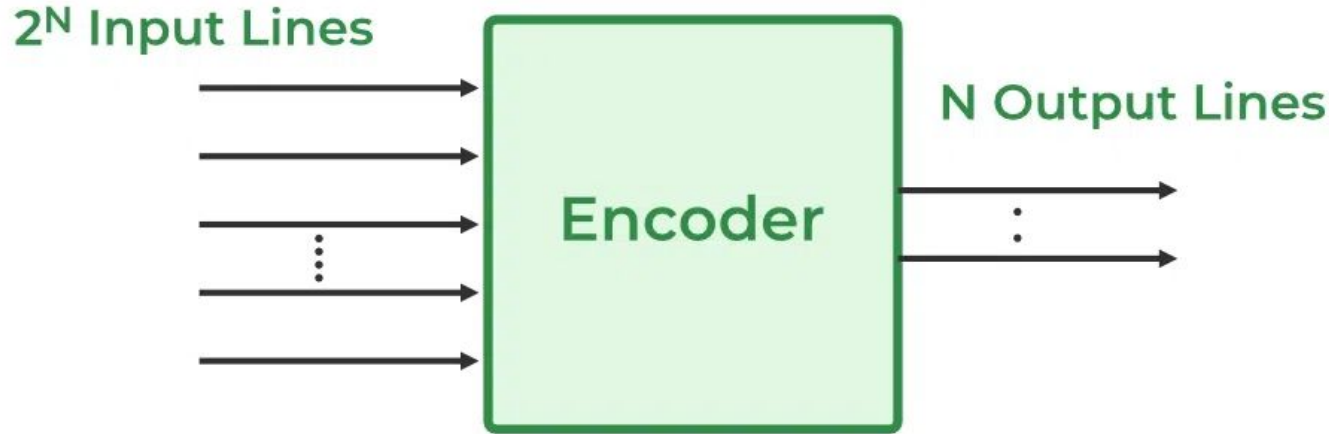
Implementation of LUTs as MUXs

Truth Table

x_1	x_2	x_3	f
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

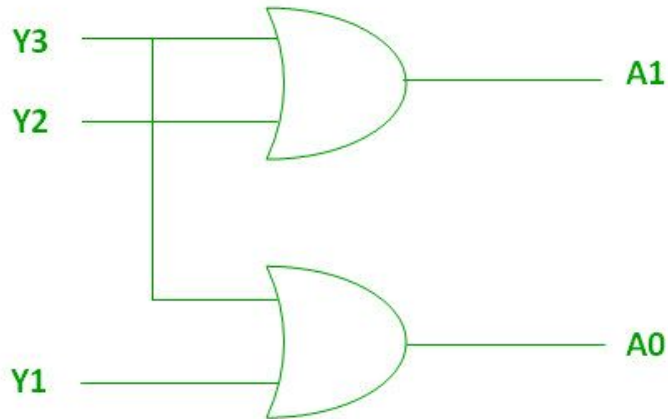


Encoder



- The number of inputs is usually 2^n , with n output lines.
- Inputs are usually active high, meaning high voltage represents an active signal.
- Binary-weighted encoders assign codes based on input position using binary values.
- Encoders help convert parallel inputs into compact binary formats for processing.

4:2 Encoder



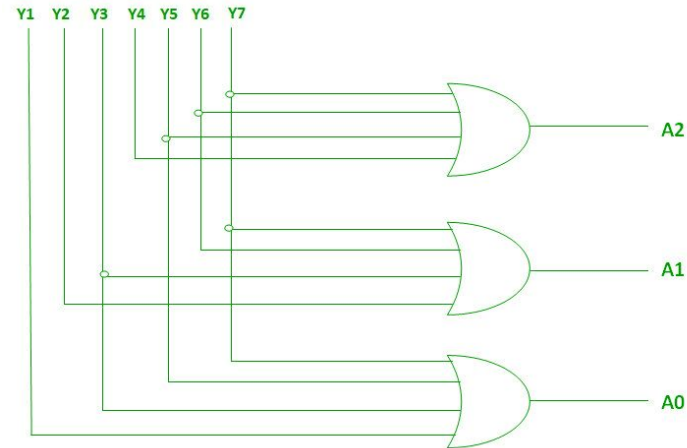
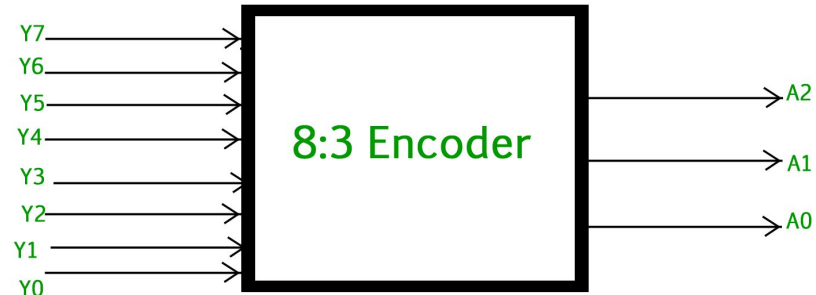
Can you implement this using mux?

INPUTS($Y_3Y_2Y_1Y_0$)	OUTPUTS(A_1A_0)
0001	00
0010	01
0100	10
1000	11

$$A_0 = Y_3 + Y_1$$

$$A_1 = Y_3 + Y_2$$

8:3 Encoder



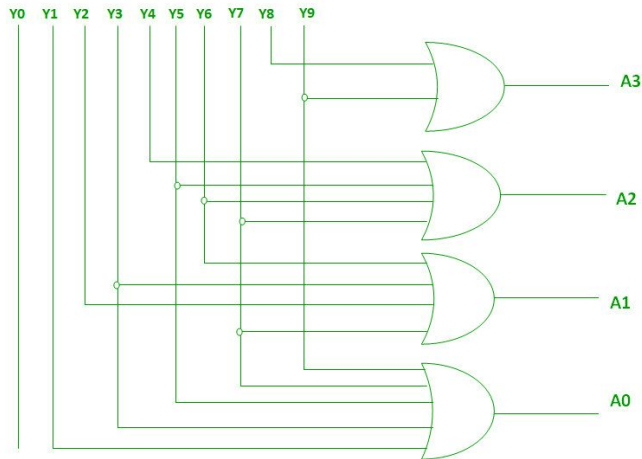
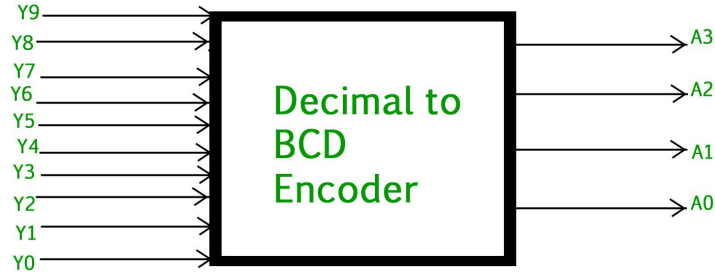
INPUTS($Y_7Y_6Y_5Y_4Y_3Y_2Y_1Y_0$)	OUTPUTS($A_2A_1A_0$)
00000001	000
00000010	001
00000100	010
00001000	011
00010000	100
00100000	101
01000000	110
10000000	111

$$A_0 = Y_7 + Y_5 + Y_3 + Y_1$$

$$A_1 = Y_7 + Y_6 + Y_3 + Y_2$$

$$A_2 = Y_7 + Y_6 + Y_5 + Y_4$$

Decimal to BCD Encoder



INPUTS(Y ₉ Y ₈ Y ₇ Y ₆ Y ₅ Y ₄ Y ₃ Y ₂ Y ₁ Y ₀)	OUTPUTS(A ₃ A ₂ A ₁ A ₀)
0000000001	0000
0000000010	0001
0000000100	0010
0000001000	0011
0000010000	0100
0000100000	0101
0001000000	0110
0010000000	0111
0100000000	1000
1000000000	1001

$$A_0 = Y_9 + Y_7 + Y_5 + Y_3 + Y_1$$

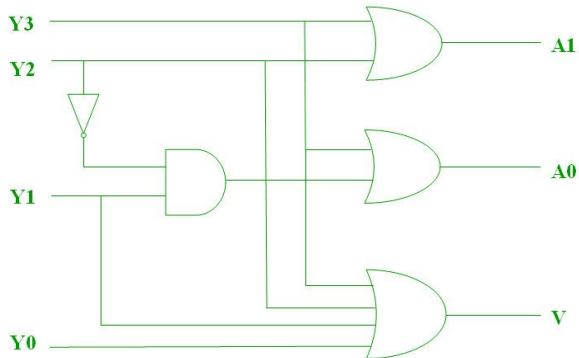
$$A_1 = Y_7 + Y_6 + Y_3 + Y_2$$

$$A_2 = Y_7 + Y_6 + Y_5 + Y_4$$

$$A_3 = Y_9 + Y_8$$

4:2 Priority Encoder

INPUTS($Y_3Y_2Y_1Y_0$)	OUTPUTS(A_1A_0V)
0000	XX0
0001	001
001X	011
01XX	101
1XXX	111



$Y_1 Y_0$		00	01	11	10
$Y_3 Y_2$	00	X	0	0	0
	01	1	1	1	1
	11	1	1	1	1
	10	1	1	1	1

$$A1 = Y3 + Y2$$

$Y_1 Y_0$		00	01	11	10
$Y_3 Y_2$	00	X	0	1	1
	01	0	0	0	0
	11	X	X	X	X
	10	1	1	1	1

$$A0 = Y3 + Y2' Y1$$

Errors in Encoders

There are some errors that usually happen in Encoders are mentioned below:

- There is an ambiguity, when all outputs of the encoder are equal to zero.
- If more than one input is active High, then the encoder produces an output, which may not be the correct code.

So, to overcome these difficulties, we should assign priorities to each input of the encoder. Then, the output of the encoder will be the code corresponding to the active high inputs, which have higher priority.

8:3 Priority Encoder

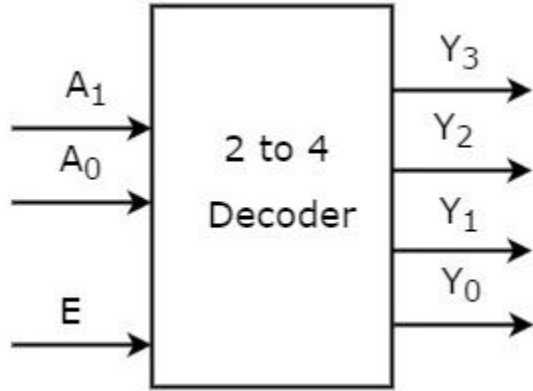
Inputs								Outputs		
D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	Q_2	Q_1	Q_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	x	0	0	1
0	0	0	0	0	1	x	x	0	1	0
0	0	0	0	1	x	x	x	0	1	1
0	0	0	1	x	x	x	x	1	0	0
0	0	1	x	x	x	x	x	1	0	1
0	1	x	x	x	x	x	x	1	1	0
1	x	x	x	x	x	x	x	1	1	1

$$Q_0 = D_7 + D_5 \overline{D_6} \overline{D_7} + D_3 \overline{D_4} \overline{D_5} \overline{D_6} \overline{D_7} + D_1 \overline{D_2} \overline{D_3} \overline{D_4} \overline{D_5} \overline{D_6} \overline{D_7}$$

$$Q_1 = D_6 + D_7 + (D_2 + D_3) \overline{D_4} \overline{D_5} \overline{D_6} \overline{D_7}$$

$$Q_2 = D_4 + D_5 + D_6 + D_7$$

2:4 Decoder



$$Y_3 = E \cdot A_1 \cdot A_0$$

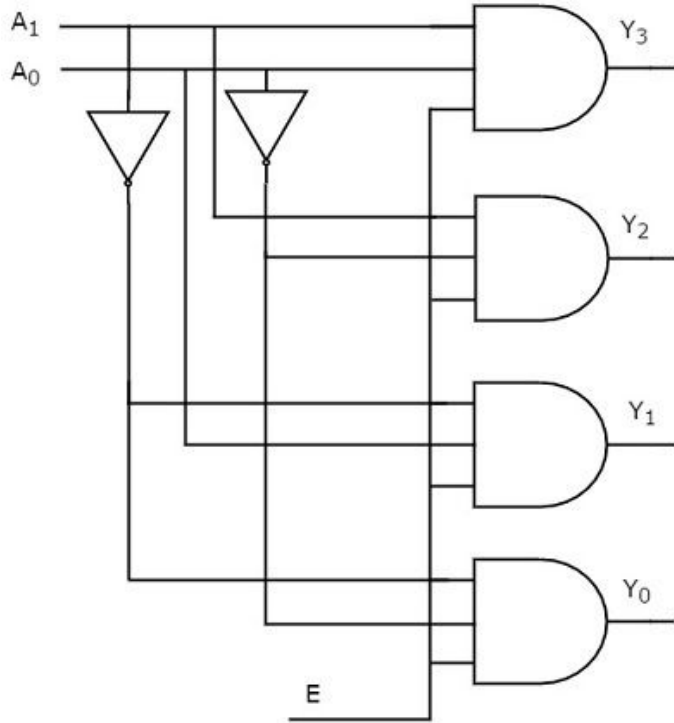
$$Y_2 = E \cdot A_1 \cdot A_0'$$

$$Y_1 = E \cdot A_1' \cdot A_0$$

$$Y_0 = E \cdot A_1' \cdot A_0'$$

E	A1	A0	Y3	Y2	Y1	Y0
0	x	x	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

2:4 Decoder



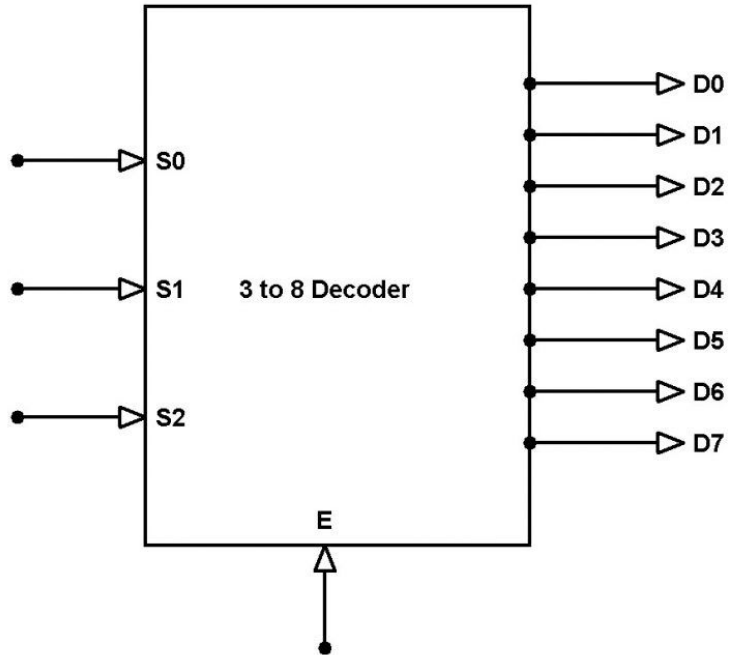
$$Y_3 = E \cdot A_1 \cdot A_0$$

$$Y_2 = E \cdot A_1 \cdot A_0'$$

$$Y_1 = E \cdot A_1' \cdot A_0$$

$$Y_0 = E \cdot A_1' \cdot A_0'$$

3:8 Decoder



A	B	C	D0	D1	D2	D3	D4	D5	D6	D7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	1	0	0	0	0	0	0	1

3:8 Decoder

$$D0 = A'B'C'$$

$$D1 = A'B'C$$

$$D2 = A'BC'$$

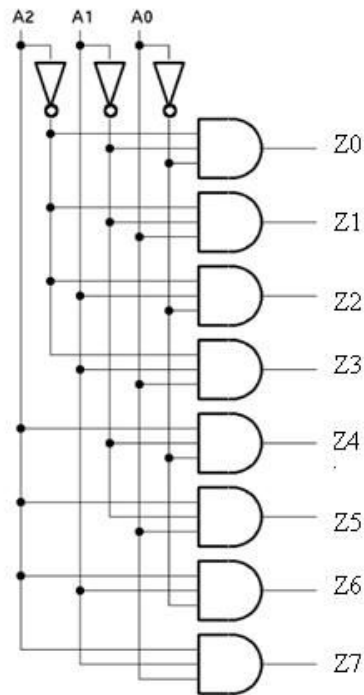
$$D3 = A'BC$$

$$D4 = AB'C'$$

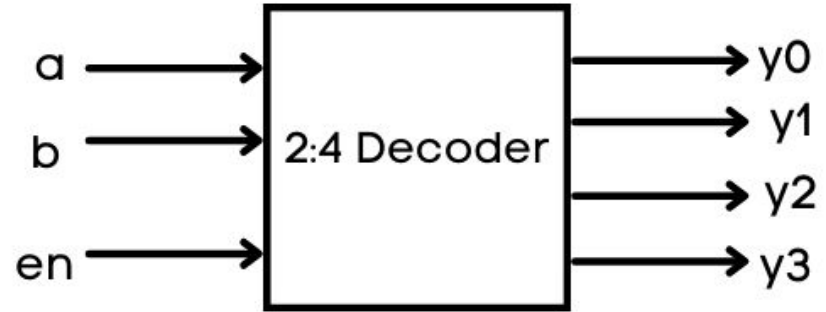
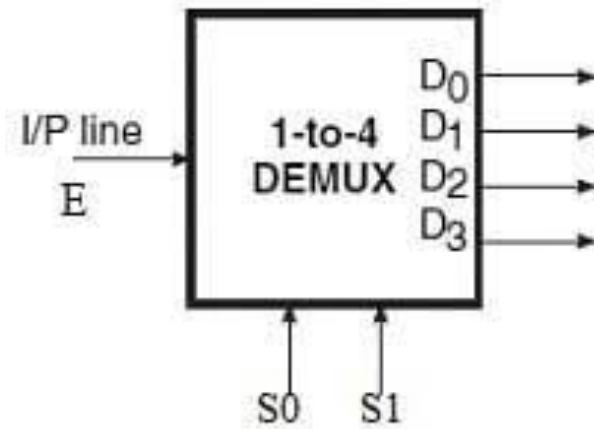
$$D5 = AB'C$$

$$D6 = ABC'$$

$$D7 = ABC$$



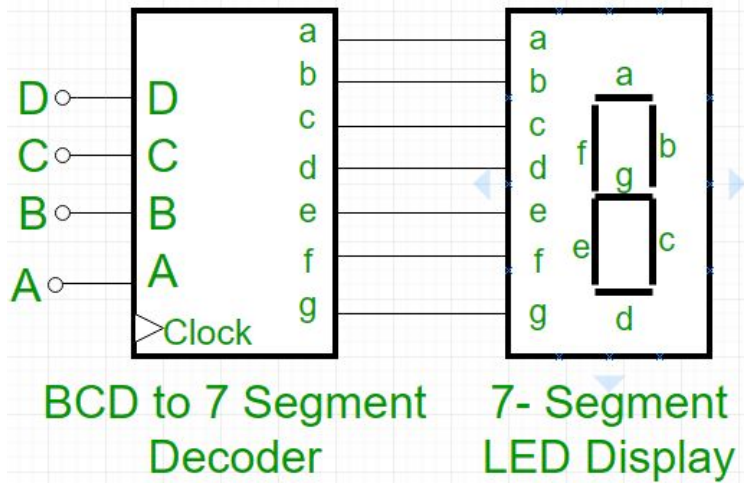
Decoder and Demux



A decoder can be implemented using a demultiplexer:

- The input pins of the decoder is the select pins of the demultiplexer
- The enable (E) pin of the decoder is the input pin of the demux

BCD to Seven (7) Segment Display



A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1

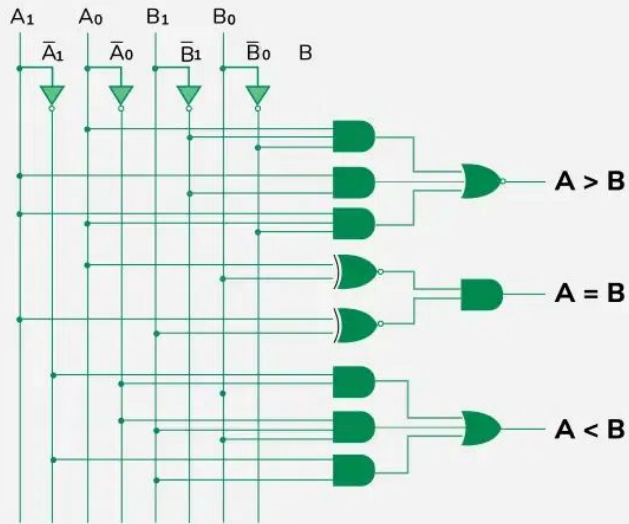
BCD to Seven (7) Segment Display

Segment a	$A + C + (B \oplus D)$
Segment b	$\bar{B} + (C \oplus D)$
Segment c	$B + \bar{C} + D$
Segment d	$A + (A + \bar{B}C) + (\bar{B}D) + (C\bar{D}) + (B\bar{C}D)$
Segment e	$(\bar{B}D) + (\bar{C}D)$
Segment f	$A + (\bar{C} \cdot \bar{D}) + (B \cdot \bar{C}) + (BD)$
Segment g	$A + (B\bar{C}) + (\bar{B}C) + (CD)$
Segment g	$A + (B\bar{C}) + (\bar{B}C) + (\bar{C}D)$

Comparator



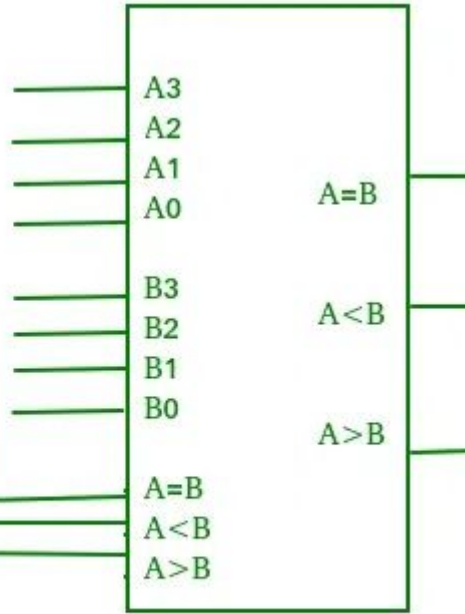
2 Bit Magnitude Comparator



INPUT				OUTPUT		
A1	A0	B1	B0	A<B	A=B	A>B
0	0	0	0	0	1	0
0	0	0	1	1	0	0
0	0	1	0	1	0	0
0	0	1	1	1	0	0
0	1	0	0	0	0	1
0	1	0	1	0	1	0
0	1	1	0	1	0	0
0	1	1	1	1	0	0
1	0	0	0	0	0	1
1	0	0	1	0	0	1
1	0	1	0	0	1	0
1	0	1	1	1	0	0
1	1	0	0	0	0	1
1	1	0	1	0	0	1
1	1	1	0	0	0	1
1	1	1	1	0	1	0

Comparator

8 bit comparator using two cascaded 4 bit comparators



4 bit comparator

